Can Deterministic Network Verification Reduce Undetected Faults in LLM-Generated Configurations?

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

***Abstract*—We preregistered and executed a paired confirmatory study (study id c1-gnr-vv-2026-0001) to measure whether inserting a deterministic network verifier between an LLM intent→configuration generator and an emulated execution reduces the proportion of configurations that would execute with operational faults. Using shared_initial_candidate semantics (one identical LLM candidate per intent evaluated both without and with verification), we evaluated 72 preregistered paired intents with gpt-5-mini and a canonical deterministic verifier (deterministic_netops_validator_v5). Execution completed with 144 episodes (72 pairs) and 72 model calls. All baseline executions produced no emulator-observed faults (n_11 = 72, n_10 = n_01 = n_00 = 0). The paired difference in undetected-fault proportion is 0.0 (paired bootstrap 95% CI [0.0, 0.0], exact McNemar two-sided p = 1.0; bootstrap seed = 1729, resamples = 10000). Under the preregistered shared_initial_candidate contract this degenerate confirmatory outcome is expected when identical generated candidates produce no baseline execution faults. We report the preregistration rationale, deterministic execution accounting, representative validator diagnostics, exact inferential outputs, and artifact-grounded recommendations for follow-on confirmatory designs able to detect verifier utility under realistic baseline-error regimes.**

I. Introduction

Generative large language models (LLMs) are increasingly proposed to translate high-level operator intents into device-level network configurations. This intent→configuration automation can speed change pipelines but risks silently violating reachability, access-control, ordering, or workflow invariants and thereby causing outages if generated text is executed without additional checks. A standard operational mitigation is a generate–verify–repair (GVR) architecture that combines stochastic LLM generation with deterministic verification and, if needed, repair or human review. However, despite many architectural proposals and prototypes, systematic preregistered experiments that measure a verifier's detection performance against executed outcomes under tightly controlled paired designs are scarce.

This paper answers a narrowly scoped, preregistered research question: does inserting a deterministic network verifier between an LLM generator and emulator execution materially reduce the proportion of configurations that execute with operational faults when the identical generated candidate is evaluated both without and with verification? That estimand isolates verifier detection from improvements that could arise from re-sampling, prompt-engineering, or repair-enabled iterations. We executed study c1-gnr-vv-2026-0001 using gpt-5-mini and deterministic_netops_validator_v5 across 72 preregistered paired intents under shared_initial_candidate semantics. The run completed with no technical failures, produced a degenerate confirmatory outcome (no baseline emulator faults), and yields a paired difference of 0.0 (95% CI [0.0,0.0], exact McNemar p = 1.0). Rather than treating this as a null failure, we analyze why it occurred given the preregistered contract and archived artifacts, and we provide concrete, artifact-grounded guidance for designs that can detect verifier utility when baseline error prevalence is non-negligible.

II. Related Work

Work on LLM-driven NetOps spans intent→configuration translation, multi-agent/tool-augmented orchestration, and architectural proposals for combining stochastic generation with deterministic checks. Prior empirical studies demonstrate that modern LLMs can produce device-level configurations from high-level intent in constrained vendor-dialect settings and curated evaluation suites; these results show promise but are typically scoped to limited task families and curated datasets rather than broad production traces [1]. That line of work motivates leveraging LLMs to reduce operator effort while highlighting the need for safety controls when configuration semantics and ordering matter.

Complementing single-agent generation work, multi-agent and tool-augmented frameworks have been proposed that explicitly integrate verifiers, tool calls, or specialized modules into LLM-driven NetOps pipelines to reduce regressions and coordinate multi-step changes [2]. These systems illustrate practical architectures for inserting deterministic checks or domain services into agentic workflows, and their evaluations commonly rely on emulators, curated scenarios, or prototype deployments. Such prototypes suggest feasible integration points for verifiers but do not, in the supplied records, provide preregistered paired measurements of verifier detection performance against executed faults.

A related and more general strand of work articulates and evaluates generate–verify–repair (GVR) patterns outside NetOps—particularly in code- and software-repair domains—where stochastic generators produce candidates that deterministic checkers validate and, when necessary, trigger repair iterations [3]. These studies document how deterministic validation can convert nondeterministic generation into a correctness-convergent workflow; they also show that the empirical util-

ity of verification depends strongly on the baseline error rate of generator outputs and on whether the architecture permits verifier-triggered repairs or resampling. Translating these insights into NetOps requires coupling LLM outputs to domain-appropriate verifiers and measuring detection relative to executed outcomes.

Canonical deterministic network-verification techniques—exemplified by header-space analysis and related formal reachability/policy analyzers—provide precise, semantics-aware invariants for reachability and ACL-style policy checks and are natural candidates for the 'verify' stage in GVR architectures [4]. Such verifiers can deterministically flag violations of specified invariants or workflow constraints expressed over packet headers, interface states, and rule orderings. However, the literature lacks, among the supplied records, preregistered, paired experiments that measure how often these canonical verifiers would have detected faults that would otherwise manifest when LLM-generated configurations are executed in an emulator or live device.

Taken together, the verified literature establishes three pertinent points: (i) LLMs can produce plausible device-level configs from intent in constrained settings [1]; (ii) system proposals and prototypes show how verifiers and tools can be integrated into multi-agent NetOps workflows [2]; and (iii) GVR patterns can yield empirical correctness improvements in other domains when baseline generator errors are non-negligible and when verifier-triggered repairs or resampling are allowed [3]. What remains underexplored in the supplied evidence is a preregistered, paired measurement that isolates verifier detection on identical candidates—i.e., does a canonical deterministic verifier reduce the probability of an execution fault for the same LLM-generated candidate when the candidate is executed with and without prior verification? Our study directly targets this measurement gap by adopting a shared_initial_candidate paired design and reporting exact execution-accounting artifacts to make that detection question reproducible and auditable.

III. Methodology

Preregistration and primary estimand. We preregistered and froze study c1-gnr-vv-2026-0001 before observing outcomes. The primary estimand is the paired reduction in undetected operational-fault proportion (paired_success_rate_difference_guarded_minus_baseline).
The confirmatory hypothesis (H1) targeted an absolute paired reduction of 0.20 (two-sided alpha = 0.05, power >= 0.80); a sample-size heuristic yielded ≈63 pairs and we preregistered N = 72 pairs (24 per topology-difficulty stratum). The preregistration and analysis plans, including sensitivity analyses and exclusion rules, are part of the archived preregistration record.

Shared_initial_candidate semantics and experimental contract. The execution_contract enforced shared_initial_candidate semantics: for each intent we performed exactly one initial LLM generation and evaluated that identical candidate in two paired conditions. Baseline condition: execute the shared candidate in an isolated emulator and record whether emulator-observed operational faults occur. Guarded condition: run the canonical deterministic verifier on the same candidate; if the verifier flags violations the guarded outcome is recorded as prevented (no executed fault); if the verifier does not flag, execute the identical candidate in the emulator and record the result. By construction, any observed paired difference can only arise from the verifier preventing execution of a candidate that would otherwise have failed at emulation; this isolates detector utility from generator-side resampling or repair effects.

Model, adapter, and verifier configuration (artifact-grounded). The declared model is gpt-5-mini (execution/model_configuration.json) requested via the hosted_netops_gvr_v1 adapter. Key recorded model parameters: declared_model_version = "gpt-5-mini", provider = "openai_responses", max_output_tokens = 2000, maximum_attempts_per_call = 1. The recorded temperature = null indicates that provider-default runtime sampling semantics were used; we therefore treat sampling as provider-default rather than as a fixed numeric temperature. The canonical verifier is deterministic_netops_validator_v5 configured to run the full_intent_constraint_validation rule set. The verifier emits structured validator_traces per episode (parsed_command_count, required_command_count, normalized_configuration, validation_before/after snapshots, violation_codes, violation_count, and difficulty metadata). The verifier's validity verdict and trace for each episode were archived and formed the mechanistic basis for guarded decisions.

Task-bank construction, contamination controls, and stratification. Confirmatory tasks were generated deterministically by deterministic_netops_task_generator_v5. Each task_manifest entry encodes a natural-language intent, a machine-structured required_sequence (ordered field changes needed to implement the intent), allowed_touched_fields, preserved-state policies, initial and expected_final_state snapshots, and a difficulty.level tag (medium/hard/very_hard). The preregistration required deterministic exact-match contamination checks against a public-corpus fingerprint; exact-match flagged items are excluded from confirmatory analysis (none were excluded in this run). The confirmatory sample followed the preregistered stratification (24 pairs per difficulty stratum) to allow descriptive subgroup inspection.

Execution protocol, retries, and deterministic accounting. The missingness_plan permitted up to two retries (three attempts total) for transient hosted-API errors on generation calls; all retries and provider events were logged. The execution manifest records planned_episode_count = 144, completed_episode_count = 144, failed_episode_count = 0, and model_calls_used = 72 (one shared generation per pair). Provider-call audit reports hosted_model_call_event_count = 72, completed_hosted_model_call_event_count = 72, failed_hosted_model_call_event_count = 0, cache_miss_count = 72, and stage_counts = {"shared_initial_generation": 72}.

Scoring, validation rules, and per-episode traces. For

each episode the verifier produced a binary validity verdict and an accompanying validator_trace JSON that documents parsed and required command counts, normalized_configuration, validation_before/after states, and any violation_codes. The scoring rule deployed was full_intent_constraint_validation (validator enforces required_sequence order, allowed_touched_fields, and preserve_unspecified_state constraints encoded in the task manifest). Per-episode scoring artifacts therefore permit audit of why a candidate was accepted or flagged. In the recorded run no verifier flagging occurred for confirmatory pairs (violation_count = 0 across representative episodes), and no automated repair was applied (repair_applied = false in validator_trace fields).

Inferential and sensitivity analysis procedures. The primary analysis used the registered paired_binary_analysis_v1 executor to construct the 2×2 contingency table (guarded × baseline) using binary indicators where an undetected executed fault is 1 and success/no-fault is 0. The prespecified exact inferential test is an exact McNemar two-sided test when discordant pairs exist. We also computed paired bootstrap-percentile 95% confidence intervals for the paired difference using 10,000 resamples (bootstrap_seed = 1729). Prespecified subgroup confirmatory comparisons (topology complexity and policy type) were registered with Holm correction to control familywise error; these controls are implemented in the deterministic analysis pipeline. Pre-registered sensitivity analyses include (a) treating excluded confirmatory pairs as failures and (b) bootstrap diagnostics for detector detection and false-positive rates. All analysis was executed by deterministic scripts whose outputs are archived with the study.

IV. Results

Execution and manifest summary. The autonomous pipeline completed without technical failures. Key manifest figures: planned_episode_count = 144; completed_episode_count = 144; failed_episode_count = 0; model_calls_used = 72; artifact_count = 510. analysis/condition_summary.csv reports baseline successes = 72, baseline failures = 0 (success_rate = 1.0) and guarded successes = 72, guarded failures = 0 (success_rate = 1.0).

Paired contingency and exact inference. Aggregating across the 72 confirmatory pairs produced contingency counts n_11 = 72, n_10 = 0, n_01 = 0, n_00 = 0 (guarded × baseline). The paired estimate paired_success_rate_difference_guarded_minus_baseline = 0.0. Exact McNemar two-sided p = 1.0. The paired bootstrap percentile 95% CI (seed = 1729; 10,000 resamples) is [0.0, 0.0]. analysis/paired_contingency_table.csv and analysis/condition_summary.csv contain these tabulations and are archived.

Representative validator diagnostics and per-episode exemplars. Representative per-episode scoring artifacts are archived for audit. Representative summaries (extracted from archived JSONs): pair-000001 (task-000001): difficulty.level = "medium", parsed_command_count = 4, required_command_count = 4, violation_count = 0; pair-000002 (task-000002): difficulty.level = "hard", parsed_command_count = 2, required_command_count = 2, violation_count = 0; pair-000003 (task-000003): difficulty.level = "hard", parsed_command_count = 6, required_command_count = 6, violation_count = 0. These traces show the verifier executed on each shared candidate and produced no violation flags across representative tasks.

Contamination, missingness, and sensitivity diagnostics. analysis/contamination_summary.csv reports zero exact-match contaminations for confirmatory pairs. analysis/missingness_summary.csv records complete_pairs = 72 and zero failed or unscorable episodes. Pre-specified sensitivity analyses (treating excluded pairs as failures; bootstrap diagnostics) were executed and archived; because no pairs were excluded and no discordant outcomes occurred these sensitivity analyses reproduce the degenerate numeric conclusion (paired difference = 0.0). Bootstrap diagnostics used seed = 1729 and 10,000 resamples.

V. Discussion

Confirmatory interpretation and boundary conditions. The preregistered paired design with shared_initial_candidate semantics validly measures verifier detection on identical candidates; because the identical generated candidates produced zero emulator faults the verified paired outcome is a ceiling/null result: the verifier could not reduce executed faults that did not occur. The numeric outputs (n_11 = 72; paired difference = 0.0; McNemar p = 1.0; bootstrap CI [0.0,0.0]) follow directly from the preregistered contract and observed data. Reporting this degenerate outcome documents a reproducible case where the preregistered estimand and data-generating conditions leave no room for the verifier to demonstrate utility.

Artifact-grounded diagnostic factors. Archived artifacts allow concrete attribution of this ceiling outcome to measurable pipeline factors. First, the deterministic task generator produced structured manifests with explicit required_sequence and allowed_touched_fields that constrain acceptable configs; these manifest constraints increase the probability of a single correct generation. Second, the declared model produced candidates that consistently matched those structured constraints across the synthetic corpus (shared_initial_candidate traces). Third, the verifier's rule set (full_intent_constraint_validation) enforces the same required_sequence and field constraints encoded in the manifests; per-episode validator_traces show parsed_command_count = required_command_count and violation_count = 0 for representative tasks. Together these aligned design choices—constrained intents, congruent verifier rules, and a model that produced matching sequences—explain the absence of baseline emulator faults.

Design-space alternatives to detect verifier utility. A paired confirmatory test that targets a fixed absolute reduction requires a non-zero baseline prevalence of executed faults to have power to detect a reduction; when baseline prevalence is zero the paired difference must be zero regardless of verifier performance. This algebraic consequence under

shared_initial_candidate semantics implies that confirmatory designs intended to measure verifier detection should ensure the task bank yields a measurable baseline error rate (e.g., by including ambiguous intents, adversarial cases, or known failure-mode seeds) or should permit condition-specific sampling or repair so that the verifier can alter the executed candidate distribution.

Concretely, the archived pipeline supports several preregisterable modifications that change the estimand and enable non-degenerate inference: (1) independent-generation arms (separate initial generator calls per condition) convert the estimand to a comparison of execution outcomes when generation itself may differ across conditions; (2) repair-enabled flows (permit a deterministic repair when the verifier flags a violation and then execute the repaired candidate) measure the end-to-end benefit of verification-plus-repair; (3) adversarial/fault-injection task banks intentionally raise baseline error prevalence to power detector effect estimation; (4) explicit sampling-parameter sweeps and multi-model comparisons measure sensitivity to generator stochasticity; and (5) dedicated false-positive cost arms quantify the operational blocking cost of verifier flags. Each alternative is preregisterable and implementable with the same evidence-recording conventions used here; choosing among them determines whether the estimand isolates detector-only detection (shared_initial_candidate) or measures practical net benefit including generator-side adaptations (independent generation, repair).

Threats to validity revisited. Internal validity: shared_initial_candidate semantics isolates verifier detection but prevents verifier-mediated resampling or repair effects; the provider-call audit confirms one shared generation per pair. Construct validity: emulator-executed faults are a conservative surrogate for operational harm but do not capture traffic-dependent timing or vendor-hardware idiosyncrasies. External validity: the run used a single declared model (gpt-5-mini) and a synthetic task bank; extrapolation to other models, broader real-world distributions, or production hardware is limited. Conclusion validity: with zero baseline faults the preregistered power analysis targeted to an absolute 0.20 reduction becomes moot; future confirmatory designs must align expected baseline prevalence with power targets.

Operational implications and measured tradeoffs. The observed ceiling does not imply that verifiers are unnecessary in practice. Instead, it shows that under constrained, high-clarity intents and a verifier rule set that encodes the same constraints, an LLM can produce device-level sequences that pass deterministic checks and emulator execution. Measuring the verifier's net operational value requires explicit, preregistered trade-off quantification: estimate true-positive detection rates on a task bank with non-negligible baseline faults, and quantify verifier false-positive blocking costs. The archived artifacts and deterministic accounting in this study provide a reproducible template to design such targeted, powered follow-on experiments.

VI. Limitations

- Confirmatory ceiling/null outcome: the baseline generator produced zero emulator faults on the preregistered task set, preventing direct measurement of verifier detection benefit.
- Shared_initial_candidate semantics: the design measures verifier effect on the same generated candidate only and does not assess independent per-condition generation, multi-sample generation, or repairs unless explicitly permitted by the contract.
- Single-model scope: experiments used one declared model (gpt-5-mini); results do not imply generality across models or versions.
- Synthetic/emulated tasks: task corpus and emulator executions differ from production devices and live traffic; external validity to production deployments is limited.
- Sampling-parameter uncertainty: execution metadata records temperature = null (sampling parameters unset); null indicates provider-default sampling semantics and is recorded as residual uncertainty.
- Residual contamination risk: deterministic provenance and exact-match checks were applied, but private training-data contamination of the hosted model cannot be fully ruled out and remains residual uncertainty.

VII. Conclusion

We executed a preregistered paired evaluation (c1-gnr-vv-2026-0001) under shared_initial_candidate semantics to test whether a canonical deterministic verifier reduces the proportion of LLM-generated configurations that would execute with operational faults. For the chosen model (gpt-5-mini), verifier (deterministic_netops_validator_v5), and synthetic/emulated task bank, baseline executions produced zero emulator faults and the verifier did not change outcomes (paired difference = 0.0; bootstrap 95% CI [0.0,0.0]; exact McNemar p = 1.0). This artifact-grounded null/ceiling outcome is internally consistent with the preregistered design: when identical generated candidates produce no executed faults a verifier cannot reduce executed faults under the shared_initial_candidate contract. Measuring verifier utility under realistic error regimes requires preregistered contracts that permit independent or multiple generator samples, repair-enabled flows, adversarial task sets, and multi-model comparisons. The archived artifacts (responses, task manifests, validator traces, execution manifest, and analysis logs) provide a reproducible baseline and a template for those follow-on confirmatory designs and power calculations.

Disclosure Statement

Disclosure: Human role and autonomy boundary—A human Research Director selected the broad topic family and approved the immutable initial master prompt before the autonomy lock; the study execution, analysis, and manuscript text were produced autonomously after that lock with no human manual editing of technical content.

Models and orchestration—initial generation used gpt-5-mini (declared model version recorded in execution metadata) orchestrated via the hosted_netops_gvr_v1 adapter; deterministic_netops_validator_v5 served as the canonical verifier; an isolated emulator executed candidate configurations. Preregistration and archives—study c1-gnr-vv-2026-0001 was preregistered and frozen prior to execution; execution and analysis manifests and raw artifacts are archived with the study (see the execution manifest in the archived bundle). Immutable master-prompt reference (archived): provenance/master_prompt.txt (SHA-256: 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39b15ba).
No human manually edited this manuscript after the autonomy lock.

REFERENCES

[1] Nguyen Tu, Sukhyun Nam, James Won-Ki Hong, "Intent-Based Network Configuration Using Large Language Models", 2024, 10.1002/nem.2313

[2] Zhaodong Wang, Samuel Lin, Guanqing Yan, Soudeh Ghorbani, Minlan Yu, Jiawei Zhou, Nathan Hu, Lopa Baruah, Sam Peters, Srikanth Kamath, Jian Yang, Ying Zhang, "Intent-Driven Network Management with Multi-Agent LLMs: The Confucius Framework", 2025, 10.1145/3718958.3750537

[3] Rafael Cadenas, "Convergent Correctness in Stochastic Code Generation: A Generate-Verify-Repair Architecture for Deterministic Validation of Autonomous AI Coding Agents in Regulated Environments", 2026, 10.2139/ssrn.6754899

[4] http://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.300.8659